

ONLY PERSONS WITH
ID-C5 CARD

T3
MODDING GUIDE

A Beginner's Handbook for Installing and Creating Mods with T3-ModTools.

EDITION 1.2 | AUGUST 2026
Based on T3-ModTools 0.6.12.

T3-ModTools is experimental.
Keep original files and your own work backed up.

Contents

Use the linked contents to navigate to chapters. This edition covers T3-ModTools 0.6.12. The workshops teach reusable workflows rather than the configuration of a particular released mod.

How to Use This Book
Scope and learning projects
Quick Glossary
Part I - Install and Play Mods Safely
1. What T3-ModTools Does
2. Install T3-ModTools
3. Install mod projects
4. Understand Load Order and Conflicts
5. Verify the Installation In Game
6. Update, Disable, or Remove Mods
Part II - Extract and Understand Game Assets
7. Asset Extraction Mode
8. Choose the Right Output
9. Axes, Mirroring, UVs, and Winding
10. Find the Internal Path That Matters
Part III - Create, Build, and Test Your Own Mod
11. Create a Mod Project
12. Make a First Exact-Path Replacement
13. Build Discipline and Versioning
14. Troubleshoot the Build
Part IV Asset editing workshops
15. Workshop: Weapon and Vehicle Sound Replacement
16. Workshop: Replace Music
17. Workshop: Edit Weapon Balance
18. Workshop: Plasma Projectile, Muzzle, and Impact Effects
19. Workshop Character models textures materials and menus
Part V - Blender Workflows
20. Install the Blender Plug-ins
21. Rigged SCA Replacement Workflow
22. LOD Import and Export
23. Animation Import and Export
24. T3 Transform Converter
Part VI - Executable Patches and Advanced Research
25. Executable Patch Manifests
26. Known Limits and Research Frontiers
Part VII - Reference
27. Folder and Path Reference
28. Pre-Build Checklist
29. Post-Build Checklist
30. Failure-to-Cause Map
31. A Repeatable Modding Method
Part VIII Extended workshops for ModTools 0 6 12
32 Adding a new weapon
33 Adding a new playable or AI unit
34 Designing weapon audio and audiovisual effects
35 Choosing and using the Blender plugins
36 Building reliable generators and dependencies
37 Incremental builds packs and diagnostics
38 Testing and distributing new content
Source Notes

For T3-ModTools 0.6.12 | English edition | 3 November 2026

T3 Modding Handbook

How to Use This Book

This handbook teaches Terminator 3 War of the Machines modding with T3-ModTools 0.6.12. Begin with installation, extraction and a small replacement, then progress to models, animation, audio, new weapons and new units. Advanced registration requires validated data and binary work; the tool does not automatically discover every engine registry.

The safest learning path is:

1. Read Part I and establish a clean game installation with a small test project.
2. Complete the first replacement project in Part III.
3. Choose one workshop in Part IV.
4. Move to Blender or executable patches only after ordinary file replacement feels routine.

The guide uses three evidence labels:

Label	Meaning
Verified	The workflow is implemented by the current tool and has been exercised against the reference game installation.
Recommended	A practical starting point based on the current project, but it still needs testing on the target asset.
Experimental	The feature works at a structural level or in a specific case, but is not a general solution.

T3 is an old game with a small public modding knowledge base. That makes disciplined testing especially important. Change one thing, build, test, record the result, and keep the last known good version.

Scope and learning projects

The following generic project IDs illustrate independent areas of work. They are examples, not a required download list or a statement about the current installation:

1. `example_audio`
2. `example_music`
3. `example_balance`
4. `example_character`

WARNING `load_order.json` describes the current selection. `Mods.Cod`, `T3_Modded.exe`, and the files under `modtools_data/.compiled/` describe the last build. If the selection changed after that build, those artifacts may be stale. Always run Build Mods + Modded EXE after changing enabled mods or their order.

Quick Glossary

Term	Plain-language meaning
Asset	A game file such as a texture, model, sound, animation, or configuration file.
COD archive	A ZIP-compatible package used by T3 for game data. Important examples are `Game.cod`, `Patch116.Cod`, and `Language.cod`.
Internal path	The path an asset has inside a COD archive, such as `weapons/WM20/weapon1.option`.
Replacement mod	A mod that supplies a new file at the exact internal path of an existing file.
Overlay	A later package temporarily taking priority over an earlier package without deleting the earlier file.
Mod project	A folder containing `mod.json`, `files/`, and optional executable patches.
Build	The process that combines enabled projects into `mods/Mods.Cod` and creates `T3_Modded.exe`.
SCA	A T3 model format used by many static and rigged assets.
LOD	Level-of-detail model data used to swap geometry at different viewing distances.
ANM	A native T3 animation file.

Term	Plain-language meaning
AOB patch	A byte-pattern patch that locates a unique sequence in the original executable before changing it.

Part I - Install and Play Mods Safely

1. What T3-ModTools Does

T3-ModTools 0.6.12 is an experimental Windows application with two modes:

- Asset Extraction reads T3's ZIP-compatible COD archives, copies native assets, and converts supported models into formats that are easier to inspect or edit.
- Modding creates independent projects, manages their order and enabled state, compiles their files into `mods/Mods.Cod`, and builds a separate `T3_Modded.exe`.

The original T3.exe and Ai.Dll are never overwritten. Changed builds use those pristine inputs to produce T3_Modded.exe and, when AI patches are required, A2.Dll. A verified unchanged build can reuse its existing complete output set.

The package priority model

When launched through `T3_Modded.exe`, the reference loader gives the generated package priority before the official archives:

```
Highest priority
  mods/Mods.Cod
  Language.cod
  Patch116.Cod
  Game.cod
Lowest priority
```

If `mods/Mods.Cod` contains `weapons/WM20/weapon1.option`, the game reads that version first. The original copy remains inside `Game.cod`. Removing or disabling the mod and rebuilding restores original behavior because the base archive was never edited.

This is why most T3 mods are exact-path replacements. You are not normally editing `Game.cod`; you are placing a higher-priority file at the same internal path.

What the tool can package

The compiler can include native game files while preserving their paths, including:

- textures and skins;
- SCA, LOD, and DET model files;
- WAV and OGG audio;
- ANM animations;
- weapon, character, and vehicle configuration;
- particle, decal, effect, and shader data;
- map files and related configuration.

Replacing an existing asset is the easiest kind of mod. Adding a completely new playable character, weapon, vehicle, or map can also require identifiers, menus, AI definitions, collision data, mission references, and executable research. Begin with replacement work.

2. Install T3-ModTools

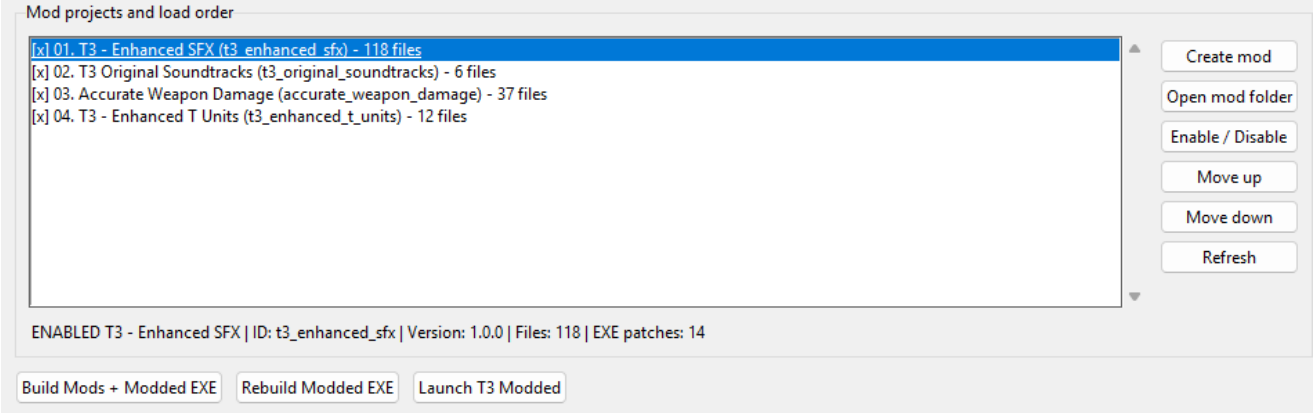
Requirements

- Windows 10 or Windows 11 for T3-ModTools.
- Python 3.9 or newer with Tkinter, which is included in the standard Windows Python installer.
- A legally owned PC copy of Terminator 3: War of the Machines.
- Blender only for optional model and animation work.

T3-ModTools itself does not require third-party Python packages.

Installation steps

1. Extract the entire `T3-ModTools` folder to a normal writable location. Do not run only one script from inside a ZIP file.
2. Double-click `T3-ModTools.bat` to start the graphical interface.
3. Select Modding mode and browse to the game folder containing T3.exe. ModTools creates the project and generated-data directories as needed.
4. Place each project directly under the selected mods folder and click Refresh.



5. Click Build Mods + Modded EXE to compile the enabled selection.
6. Once Finished this file should be created in your /T3 Root directory: T3_Modded.exe.
7. Done. To play with mods you should open the game with this launcher. You can either Launch T3_Modded.exe from the root game directory or from T3-ModTools clicking on Launch T3 Modded.

A crash is not an expected first-build step. If the generated game crashes, preserve the report, compare the pristine baseline and investigate the loader or enabled patches before continuing.

The program remembers the selected language, mode, archive path, extraction folder, game folder, and mods folder. Settings are stored in:

```
%APPDATA%\T3-ModTools\settings.json
```

Deleting that settings file resets remembered paths without deleting game data or mod projects.

3. Install mod projects

Expected game layout

```
T3 game folder/
  T3.exe
  Game.cod
  Language.cod
  Patch116.Cod
  mods/
    example_audio/
    example_music/
    example_balance/
    example_character/
```

Each project must have mod.json directly in its folder. Avoid an extra archive wrapper: use mods/example_audio/mod.json, not mods/DownloadedPack/example_audio/mod.json.

Installation steps

1. Close T3, 7-Zip, archive preview windows, and any program currently inspecting `Mods.Cod` or `T3_Modded.exe`.
2. Back up the entire game folder or at least `T3.exe` and the current `mods/` folder.
3. Copy the intended project folders into <game>/mods/.
4. Start T3-ModTools and switch to Modding.
5. Select the folder that directly contains `T3.exe`.
6. Confirm that the mods folder is `<game folder>/mods/`.
7. Refresh the project list if the folders are not shown immediately.
8. Enable the projects needed for the test and place the intended winning project highest in the list.

Order	Project ID	Main purpose
1	`example_audio`	Weapon, character, and vehicle sounds plus blue plasma projectile, muzzle, and impact work.
2	`example_music`	Six music replacements under `Musics/`.
3	`example_balance`	Selected weapon balance changes and translated option-file comments.
4	`example_character`	T-900 third-person models, LODs, material values, a Heavy texture, and menu models.

1. Click Build Mods + Modded EXE.
2. Read the result. A successful build creates `mods/Mods.Cod`, `T3_Modded.exe`, and metadata under `modtools_data/.compiled/`.
3. Click Launch T3 Modded, or launch `T3_Modded.exe` directly from the game folder.

IMPORTANT Do not launch `T3.exe` when you expect the generated `Mods.Cod` to load. `T3.exe` remains the unmodified fallback. Use `T3_Modded.exe` for the compiled setup.

4. Understand Load Order and Conflicts

Inside the mod manager, enabled projects higher in the visible list have higher priority when multiple projects provide the same internal path. Think of the list as a stack of layers: the topmost enabled provider wins.

For ordinary asset conflicts, the build records which mod supplied the winning file. This is useful when a texture or configuration change appears to be ignored.

Executable patches are stricter. If two enabled manifests try to write different bytes to the same executable location, the build stops instead of silently choosing one. Identical writes may coexist.

load_order.json

The file stores the project IDs and enabled states:

```
{
  "format_version": 1,
  "mods": [
    {"id": "example_audio", "enabled": true},
    {"id": "example_music", "enabled": true},
    {"id": "example_balance", "enabled": true},
    {"id": "example_character", "enabled": true},
    {"id": "enhanced_textures", "enabled": false},
    {"id": "dual_wm20", "enabled": false}
  ]
}
```

Use the graphical manager for routine changes. Manual edits are possible, but a typo in an ID can make a project appear missing or move it to a default position.

After an enabled-state or order change, use Build Mods + Modded EXE. Changed outputs are staged as a recoverable set, including optional AI output. An unchanged build may reuse the validated archive and binaries. The top enabled asset provider still wins; executable byte conflicts still stop the build.

5. Verify the Installation In Game

Test visible, audible, and gameplay changes separately. A useful first pass is:

1. Start `T3_Modded.exe` from the game folder.
2. Enter a local match where the edited asset, intended owner and relevant weapon are available.
3. Confirm that the title music or level music includes the replacement tracks when the corresponding cue plays.
4. Fire the WM20 and Termcannon. Check firing sound, muzzle color, projectile color, and surface impact color.
5. Inspect third-person models at near and far distances.
6. Inspect the relevant T-900 menu models.
7. Exit the game before rebuilding.

Keep a short test log. Record the build date, enabled mods, order, map, class, weapon, and result. Old games often cache state or fail in ways that look unrelated to the last edit; a written baseline saves time.

6. Update, Disable, or Remove Mods

Disable a project

1. Close the game.
2. Clear the project's Enabled checkbox in Modding mode.
3. Use the complete build action when the selection or asset set changes.
4. Run Build Mods + Modded EXE again.
5. Launch the new `T3_Modded.exe` and retest.

Remove a project

1. Disable it and rebuild once if you want a clean comparison.
2. Move the project folder out of `mods/` to a backup location.
3. Refresh the project list.
4. Rebuild.

Update a project

1. Back up the old project folder.
2. Replace the project files, keeping the same ID if it is a normal update.
3. Verify the enabled state and order.
4. Rebuild. Never assume an old `Mods.Cod` automatically includes new project files.

Return to the original game

Launch the untouched T3.exe for the original-game baseline. A build with no enabled projects is rejected; an executable-only project can produce an intentionally empty but valid Mods.Cod.

Part II - Extract and Understand Game Assets

7. Asset Extraction Mode

Asset Extraction is the discovery side of T3-ModTools. It lets you find an original file, learn its internal path, and obtain either a native copy or an editable conversion.

Basic extraction

1. Start T3-ModTools and select Asset Extraction at the top of the window.
2. Use the input selector to choose `Game.cod` from the game folder.
3. Choose a new, empty output folder outside the game installation.
4. Select only the category you need for the first pass.
5. Choose conversion options appropriate to that asset type.
6. Click Extract / Convert.
7. Read the final log and inspect the output folders.

The supported categories are Characters, Vehicles, Weapons, Maps, Menu Models, Effects, Audio, and Animations.

Base archive versus patched view

Selecting `Game.cod` gives you that archive's contents. Some official changes live in `Patch116.Cod`, and localized resources can live in `Language.cod`. When the same internal path exists in more than one official archive, the later official package is the version the normal game sees.

For a beginner replacement, begin with `Game.cod`, then check `Patch116.Cod` for the exact same path before editing. T3-ModTools can also read a compatible COD archive directly. Advanced command-line use can point at the installed game folder to build a combined view of its archives.

> **TIP** Always write down the exact archive and internal path that supplied your source. Two files with the same name can have different content or purpose.

8. Choose the Right Output

Native files

Native output is what ultimately belongs in a mod's `files/` folder. Examples include `.tga`, `.wav`, `.ogg`, `.option`, `.dat`, `.sca`, `.lod`, and `.anm`.

Converted output is a working copy for external tools. It is not automatically a valid game asset. An OBJ or glTF file must be exported back into a T3 format before the game can use it.

Static model conversion

Supported static SCA, LOD, and DET content can be converted to OBJ and MTL. OBJ is practical for geometry inspection and simple static editing, but it does not preserve a complete T3 rig or all engine-specific data.

Rigged glTF 2.0

Compatible character, vehicle, and weapon rigs can be exported to glTF 2.0. A rigged export includes:

- `model_rigged.gltf`;
- `model_rigged.bin`;
- original textures when available;
- generated PNG versions of supported DDS or TGA base textures connected to glTF materials.

UV0 is preserved as `TEXCOORD_0`. More than four vertex influences are reduced to the four strongest weights and normalized because that is the base glTF skinning layout.

Animations

The Animations category extracts original `.anm` files under `animations/raw/` and copies the Blender ANM importer into the animation output. Animation files only make sense with a compatible rig. Third-person character animations normally belong to third-person character rigs, while first-person weapon and arm animations belong to first-person rigs.

9. Axes, Mirroring, UVs, and Winding

Coordinate conversion is one of the most common reasons a model looks mirrored, sideways, invisible, or inside out.

Extraction conventions

- All converted models receive a global X mirror.
- Characters, vehicles, maps, and menu models otherwise keep the game's native Y/Z orientation.
- Weapons and effects may optionally use Z-up.
- The OBJ option Flip UV V coordinate affects OBJ output only.
- Rigged glTF always preserves the source V coordinate, regardless of that OBJ checkbox.

Practical rule

Do not apply extra flips by instinct. Record the extraction settings and let the matching exporter reverse them. Current LOD import/export plug-ins store and reuse import settings. For a glTF produced by T3-ModTools, the general SCA exporter normally uses Undo T3-ModTools X mirror.

Face winding determines which side of a triangle is visible. A mirror changes the transform determinant and can reverse winding. The current LOD and menu exporters account for the determinant. If a model disappears from one side, verify transforms and exporter settings before manually flipping every face.

UV rule

An upside-down texture usually means the V coordinate was flipped at the wrong stage. For OBJ, use the extraction checkbox consistently. For rigged glTF, do not add an OBJ-style V flip. Menu models use their specialized exporter, which preserves UV V according to the verified menu-model rules.

10. Find the Internal Path That Matters

A mod only works when its file lands at the path the game actually requests. Use this process:

1. Extract the smallest relevant category.
2. Search filenames and text references for the character, weapon, texture, or sound name.
3. Inspect small `.dat`, `.option`, `.lib`, `.sca`, and material files in a text editor.
4. Follow referenced filenames. A model may name a texture, a sound definition may name a WAV, and an animation library may name an ANM.
5. Check whether the same path appears in `Patch116.Cod`.
6. Reproduce the final path under your mod's `files/` folder.

Examples:

Goal	Internal path example
Edit WM20 primary gameplay	`weapons/WM20/weapon1.option`
Replace WM20 fire definition	`Sound44/Sound/Weapons/wm20/weapon.dat` plus matching banks
Replace T-900 Supply third-person model	`CA/casts/t900s/tps/model.sca` and `model.lod`
Replace a music cue	`Musics/palya5.ogg`
Edit traveling plasma	`Particles/FiredBullet.dat`
Edit plasma muzzle	`Particles/MuzzleFire.dat`

Preserve the spelling used by the archive, including native tokens such as `granade`. Although the compiler detects path conflicts case-insensitively, matching the original capitalization and path is the safest habit.

Part III - Create, Build, and Test Your Own Mod

11. Create a Mod Project

Use the graphical manager

1. Close the game.
2. Open T3-ModTools and switch to Modding.
3. Select the game folder containing `T3.exe`.
4. Click Create mod.
5. Enter a clear display name, author, and short description.
6. Let the tool create the safe project ID and folder structure.
7. Select the new project and click Open mod folder.

The canonical structure is:

```
mods/
  my_first_mod/
    mod.json
    files/
    patches/
```

An example `mod.json` is:

```
{
  "format_version": 1,
  "id": "my_first_mod",
  "name": "My First Mod",
  "author": "Your Name",
  "version": "1.0.0",
  "description": "A small exact-path replacement mod."
}
```

Keep the folder name and `id` stable after release. The name is for people; the ID is what `load_order.json` uses.

12. Make a First Exact-Path Replacement

A texture replacement is a good first project because it teaches the complete loop without executable work.

1. Extract the relevant Characters, Weapons, Vehicles, or Menu Models category.
2. Identify one native texture and its exact internal path.
3. Copy the original to a separate work folder.
4. Edit the copy without changing its dimensions or format on the first attempt.
5. Recreate the internal directories under your project:

```
my_first_mod/
  files/
    CA/
      textures/
        example.tga
```

1. Place the edited native texture at that path.
2. Enable only this project for the first test.
3. Click Build Mods + Modded EXE.
4. Launch the modded game and inspect the asset under predictable lighting.
5. If it works, add a second change. If it fails, compare the path, filename, format, and source archive before changing anything else.

WARNING Do not put a ZIP containing the real replacement tree inside `files/`. The compiler preserves that ZIP as one file; it does not unpack nested archives. Put the native replacement files themselves under `files/`.

13. Build Discipline and Versioning

One change per test

When learning an undocumented engine, a build with ten unrelated edits creates ten possible causes. Prefer small, named milestones:

- `1.0.0` first working replacement;
- `1.0.1` path or packaging correction;
- `1.2.0` new asset family;
- `2.0.0` incompatible redesign.

Keep source and release files separate

Prefer source/ for editable Blender files, master textures and audio sessions, with game-ready exports under files/. Keep source textures inside the owning project or packed in the .blend. Existing generators may use another documented project-local source path; never delete their editable inputs during publication.

Distribution checklist

- Include the project folder with `mod.json`, `files/`, and optional `patches/`.
- Include a readme with requirements, install order, conflicts, and test notes.
- Do not redistribute original extracted game archives or unrelated assets.
- Do not distribute your generated `T3_Modded.exe` as the preferred installation path.
- Tell users to build from their own original executable.
- Name any required T3-ModTools version.
- State whether the project has executable patches.

14. Troubleshoot the Build

Access denied or WinError 5

The usual cause is a locked file.

1. Close T3 and wait for it to exit completely.
2. Close 7-Zip, Explorer archive previews, hex editors, antivirus scans, or other tools viewing `Mods.Cod` or `T3_Modded.exe`.
3. Confirm that the game and mods folders are writable.
4. Run the build again.
5. If security software quarantines a newly generated executable, restore or allow only the local tool and file you trust. Do not disable protection globally.

A project does not appear

- Confirm `mod.json` is directly inside the project folder.
- Validate the JSON syntax.
- Confirm the folder is under the selected mods directory.
- Make sure the folder name does not begin with a dot.
- Refresh the project list.

The game launches but the mod is absent

- Confirm you launched `T3_Modded.exe`.
- Rebuild after the last enabled-state or order change.
- Check `modtools_data/.compiled/build_manifest.json` for the file and winning project.
- Compare the internal path with the extracted source.
- Check whether a higher enabled project supplies the same internal path.
- Verify that the file is a native game format, not an editing format.

The build stops on an executable patch

- The AOB pattern may have zero or multiple matches in this executable version.
- The replacement may not be the same length for `aob_replace`.
- An `aob_write` destination may fail its expected-byte validation.
- Two enabled mods may write different bytes to the same location.
- Restore a pristine original `T3.exe` and avoid using a previously modified executable as the source.

Part IV Asset editing workshops

15. Workshop: Weapon and Vehicle Sound Replacement

A firing-sound replacement can either replace a WAV at an existing path or edit the sound definition to reference another WAV. The following workshop uses native weapon paths as examples, not a specific mod configuration.

Understand the three sound banks

T3 has parallel Sound11, Sound22 and Sound44 trees. Keep equivalent intended replacements available in the banks supported by your release and test each audio configuration.

The folder name does not convert the sample. Start with the donor sample format, validate the encoded audio and test its bank selection in game.

Replace a weapon firing sound

For the WM20, the definition path is repeated as:

```
Sound11/Sound/Weapons/wm20/weapon.dat
Sound22/Sound/Weapons/wm20/weapon.dat
Sound44/Sound/Weapons/wm20/weapon.dat
```

Locate the FIRE event and record its original filename and volume. Change the WAV reference to your project-owned sample, leaving other event parameters unchanged for the first test.

Use this method:

1. Extract Audio and find the target weapon's `weapon.dat`.
2. Read the event blocks and note the relative WAV path.
3. Prepare a mono PCM WAV. For the first test, match the original sample format and keep the peak below clipping.
4. Give the file a unique, simple ASCII filename.
5. Update only the intended event's `filename` and, if necessary, `volume`.
6. Repeat the edited definition and WAV under `Sound11`, `Sound22`, and `Sound44`.
7. Preserve every other parameter until the new sound is confirmed.
8. Build and test single shots, rapid fire, indoor spaces, and several simultaneous actors.

Repeat this procedure for another weapon only after its event names and paths have been checked independently.

Automatic-fire tails and overlap

If a long tail is baked into every automatic shot, repeated events can overlap and become too loud or muddy. Current research has not established a universal per-actor single-voice rule for all T3 weapons. Treat sound design as the first solution:

- keep the repeated shot body short enough for the firing interval;
- use a restrained tail;
- test several actors firing together;
- avoid normalizing every sample to maximum loudness;
- compare in game, not only in an audio editor.

Replace a looping vehicle sound

For a vehicle, inspect its native sound definition for near and distant loop references. Replace the corresponding files without assuming that a weapon FIRE definition uses the same grammar.

For a loop:

1. Match the original channel count and format.
2. Edit at zero crossings or use a clean crossfade.
3. Keep the distant version simpler and quieter than the near version.
4. Preserve the definition's distance and behavior parameters during the first test.
5. Test approach, pass-by, idle, and departure transitions.

16. Workshop: Replace Music

Music replacement uses exact-path OGG files. The following native cue paths illustrate possible targets; choose only the cue you intend to change.

```
Musics/palya4.ogg
Musics/palya5.ogg
Musics/palya6.ogg
Musics/palya8.ogg
Musics/palya10.ogg
Musics/palya13.ogg
```

This is a good beginner project because no executable patch or registration file is needed when an existing cue is replaced.

Procedure

1. Extract Audio and locate the original OGG cue.
2. Copy the original to a backup outside the mod.
3. Export the replacement as OGG Vorbis.
4. Match the exact filename and internal path.
5. Start with a comparable channel layout, sample rate, duration, and loudness.
6. Place the file under `files/Musics/` in the mod project.
7. Enable only the music project for the first test.
8. Trigger the exact level or menu state that uses the cue.
9. Listen for start delay, abrupt ending, looping gap, distortion, and loudness relative to effects.

Replacing a file does not change when the engine chooses that cue. Changing cue selection or adding a new music slot requires following the game's configuration references and may require further research.

17. Workshop: Edit Weapon Balance

Weapon balance is authored in native option files under weapons/. Separate gameplay edits from comment translation and record the original and intended values before changing them.

Fields commonly found in weapon1.option

Field	Observed role	Editing advice
`weight`	Equipment weight.	Change only after confirming class/loadout effects.
`magazineSize`	Ammunition loaded before a reload; zero can have special behavior.	Test reload and HUD count.
`bullets`	Reserve ammunition outside the magazine.	Test spawn, pickup, and resupply.
`shootsPerMin`	Engine fire-rate parameter. Values do not resemble ordinary RPM in every file.	Treat as a game-specific parameter and test direction and timing.
`type`	Projectile family, such as `bullet`, `granade`, `missile`, or `homingMissile`.	Preserve native spellings exactly. A type change can alter more than damage.
`subType`	Weapon-category index used by other systems.	Do not change casually; particles and executable routing can depend on it.
`range`	Observed minimum and maximum effect or blast range.	Test several surfaces and targets.
`damage`	Observed minimum and maximum damage or effect strength.	Change gradually and compare target types.
`speed`	Initial projectile speed.	Test collision, visual trail, and aiming behavior.
`throw`	Spread angle in degrees according to the translated source comments.	Test standing, moving, and sustained fire.
`timeToExplode`	Immediate, delayed, or sticky countdown behavior depending on sign.	Use the original token pattern as a template.
`MRV`, `TETA`, `DMP`	Turning, inertia, and braking parameters in guided or mounted weapon contexts.	Edit one at a time.
`heatUp`	Heat or warm-up behavior.	Test sustained fire and cooldown.

Recording balance experiments

Test	Baseline	Measurement
Primary fire	Unmodified donor option file	Shots to kill and ammunition use
Secondary fire	Unmodified donor secondary behavior	Projectile and damage isolation
Sustained fire	Unmodified donor cadence	Timing heat and reload behavior

Keep a record for every changed field. Do not infer a gameplay effect from a translated comment or from a file modification date alone.

Build a small balance mod

1. Extract Weapons.
2. Copy one target `.option` file to a separate work folder.
3. Create a new mod project.
4. Recreate the path under `files/weapons/`.
5. Change one numeric field by a modest amount.
6. Preserve braces, native key names, spelling, and number order.
7. Build with only your balance mod enabled.
8. Test against a repeatable target and record shots to kill, timing, blast behavior, and HUD ammo.
9. Restore the original value before testing a different field if the first result is unclear.

TIP The token `granade` is misspelled in the native data. Keep the engine's spelling. Correcting it to `grenade` can turn a readable file into an invalid configuration.

18. Workshop: Plasma Projectile, Muzzle, and Impact Effects

The visible shot is not one effect. T3 separates at least three stages:

Stage	Reference data	What it controls
Muzzle	`Particles/MuzzleFire.dat`	Front and side muzzle textures plus muzzle light color.
Traveling projectile	`Particles/FiredBullet.dat`	Side trail and front-facing projectile textures.
Surface impact	`Particles/BulletHit.dat`, `BulletHitPlasma.dat`, and the added blue bank	Impact particles selected by weapon subtype and surface.

Changing only `FiredBullet.dat` can produce a blue bolt with a red muzzle and red impact. Test all three stages separately.

Change a projectile and muzzle consistently

1. Extract Effects.
2. Open `Particles/FiredBullet.dat` in a text editor.
3. Find the blocks for `Weapon_Wm20` and `Weapon_TermCannon`.
4. Change their side texture from the red plasma texture to a blue trail texture.
5. Change their front texture to a blue plasma front texture.
6. Add the referenced TGA files under `files/Textures/Particles/` at their exact paths.
7. Repeat the weapon-specific edits in `Particles/MuzzleFire.dat`.
8. Choose a light color consistent with the textures, then test its appearance in game. A numerical RGB value is not a universal lighting preset.
9. Build and test the muzzle and projectile before attempting impacts.

Why a new blue impact DAT is not enough

The original executable initializes a limited set of global bullet-impact banks. Creating `Particles/BulletHitPlasmaBlue.dat` does not automatically register or select it.

A dedicated impact-bank implementation needs a validated adapter that:

- allocates and initializes an independent blue `cBulletHit` bank;
- loads `Particles/BulletHitPlasmaBlue.dat` for that bank;
- routes only the intended weapon and subtype identities to the new impact IDs;
- preserves the original red plasma bank for other weapons;
- draws, updates, and releases both banks;
- preserves the twelve surface variants used by the impact system.

This is an advanced registration workflow, not automatic particle discovery. Validate every anchor and destination against the intended original binary. See Chapter 34 for a complete audiovisual test plan.

19. Workshop Character models textures materials and menus

A character appearance can span model, LOD, texture, material and menu resources. Treat each as a separately verifiable part of the replacement.

Third-person model pair

A third-person character replacement commonly includes both SCA and LOD. These native paths are examples:

```
CA/casts/t900s/tps/model.sca
CA/casts/t900s/tps/model.lod
CA/casts/t900heavy/tps/model.sca
CA/casts/t900heavy/tps/model.lod
```

Including only `model.sca` is not a complete character replacement. The game can switch to `model.lod` based on distance or code path, leaving the original or a mismatched model visible. Treat SCA and LOD as a pair.

To give a unit an independent skin, use a distinct texture path and update only the models intended to use it. Keep shared references unchanged where sharing is intentional.

A native material block may use a shader declaration such as:

```
Shader
{
    Type DIFFUSE_SPECULAR
    BaseTexture "..."
}
```

Keep the target skeleton hierarchy compatible with its animations. Bone names and parents are runtime relationships, not cosmetic labels.

Material files

T3's `.mat` files are small legacy material parameter files, not modern physically based materials. Blender roughness and metallic sliders are not automatically translated.

Compare material experiments against the donor values:

```
Material experiment record
Native path and original values
One changed field and intended effect
Map lighting and observed result
Revert or keep decision
```

Legacy material triplets are not a proven mapping to modern PBR parameters. Preserve the donor values initially, then vary one field and compare several lighting conditions.

Menu models are a separate pipeline

Examples of native menu resources include:

```
MenuDatas/3DModells/t900_posed.sca
MenuDatas/3DModells/t900_supply.sca
MenuDatas/3DModells/t900_s_illum.tga
MenuDatas/3DModells/t900_s_illum2.tga
```

Menu models use verified rules that differ from world character SCA files:

- texture library `MenuDatas/3DModells/`;
- shader type `BASE`;
- native menu-model axes;
- global X mirror undone;
- UV V preserved;
- determinant-aware winding.

Use `T3_Blender_Menu_Model_SCA_Exporter.py` for these assets. Do not use world-model settings and then repair the result with arbitrary rotations.

Recommended model test matrix

1. Test each edited class independently.
2. Test front, back, and profile views.
3. Test idle, walk, run, fire, hit, and death animations.
4. Test near, medium, and far camera distances.

5. Test bright human maps and red Terminator vision.
6. Test the menu selection and loadout screens.
7. Watch for missing faces, inverted normals, stretched UVs, wrong bone weights, and sudden LOD shape changes.

Part V - Blender Workflows

20. Install the Blender Plug-ins

The distributed Blender plugins cover canonical glTF import, SCA and LOD export, menu models, ANM animation, material controls and transform TXT reports. Chapter 35 explains every plugin and its limitations.

1. Open Blender.
2. Choose Edit > Preferences > Add-ons.
3. Click Install from Disk.
4. Select one `.py` plug-in file.
5. Enable the added plug-in.
6. Repeat for other required plug-ins.

Safe filenames do not contain dotted version suffixes. A filename such as `tool_v0.1.0.py` can be interpreted as a nested module path by some Blender versions. Keep the distributed simple filenames.

Plug-in	Use
`T3_Blender_SCA_Exporter.py`	Export a selected static or rigged replacement to SCA.
`T3_Blender_LOD_Importer.py`	Import a native model LOD template with ranges, materials, UVs, and weight maps.
`T3_Blender_LOD_Exporter.py`	Replace imported LOD levels while preserving template data.
`T3_Blender_Menu_Model_SCA_Exporter.py`	Export menu SCA files using specialized menu rules.
`T3_Blender_ANM_Importer.py`	Import an ANM onto a compatible armature.
`T3_Blender_ANM_Exporter.py`	Export the active Action back to ANM.
`T3_Transform_Converter.py`	Analyze and apply frame-to-frame bone component deltas.
T3_Blender_GLTF_Importer.py	Import rigged glTF while restoring canonical T3 mesh and instance identities.
T3CleverTools.py	Author supported T3 self-illumination and transparency properties.

These plug-ins are experimental. Keep the extracted original beside every exported replacement and test the result as a mod before overwriting any work file.

21. Rigged SCA Replacement Workflow

Import the working rig

1. In Asset Extraction, select Characters and enable rigged glTF export.
2. Locate `model_rigged.gltf` under `converted_rigged/`.
3. Use File > Import > T3 Rigged glTF to preserve canonical T3 identities. For an older glTF without metadata, supply the original SCA or LOD as the legacy template.
4. Preserve the armature, bone names, and hierarchy.
5. Do not apply an unconditional extra Y mirror to a mesh imported with the current workflow.
6. Do not separately mirror bones unless a verified target-specific correction requires it.
7. Perform an unchanged import and export comparison first. Record the importer settings and preserve alignment, hierarchy and bind relationships.

Edit safely

- Apply required mesh modifiers deliberately.
- Triangulate or validate the final mesh.
- Keep vertex weights to the target bones.
- Avoid renaming bones.
- Keep bounding proportions plausible for the original collision and animations.
- Use separate materials only when the target format and path references support them.

Export

1. Select the mesh objects. Select or link the compatible armature through an Armature modifier.
2. Use File > Export > T3 Model (.sca).
3. Set the internal texture library, for example `CA/Textures`.
4. Keep Undo T3-ModTools X mirror enabled for a mirrored glTF source whose transforms were applied.
5. Export to a work folder.
6. Inspect the SCA text before packaging.
7. Verify the target bone count, complete names, mesh and instance identity, shader type and base texture against the original. Do not impose one character template on every asset class.
8. Put the final file under the exact `CA/casts/.../tps/model.sca` path.
9. Export or preserve a compatible `model.lod` too.

Practical geometry limits

Native submesh indices are effectively 16-bit, so a submesh cannot address more than 65,536 stored vertices. The current text exporter can duplicate vertices per triangle corner. In that worst case, the practical ceiling is about 21,845 triangles per material submesh.

Recommended starting budgets for a character project are:

Level	Suggested triangle range	Purpose
LOD0	25,000 to 35,000	Near view, if exporter submesh limits are respected.
LOD1	12,000 to 18,000	Medium-near.
LOD2	5,000 to 8,000	Medium-far.
LOD3	1,000 to 2,000	Far view.

These are recommendations, not engine guarantees. Split materials and submeshes carefully and test performance on the original game's intended scale.

22. LOD Import and Export

The LOD exporter is template-based. This is valuable because a native `model.lod` can contain bones, instances, collision targets, and unedited levels that should not be rebuilt from guesses.

How LOD levels work

LOD means level of detail. A model.lod defines how the same character is shown at different viewing distances. LOD0 is the version used when the character is near the camera and normally has the most geometry and visual detail. As the LOD number increases, quality and polygon count should decrease because those versions are used farther away.

Workflow

1. Install both LOD plug-ins.
2. Import the original `model.lod` with File > Import > T3 LOD.
3. Keep the imported mesh and preview skeleton in their stored coordinate convention; do not add a universal manual bone mirror.
4. Keep the custom properties created by the importer.
5. Edit the intended LOD objects or generate lower levels with controlled Decimate modifiers.
6. Use File > Export > T3 LOD.
7. Select the original `model.lod` as the template.
8. Keep Export all LODs from the imported set enabled when the complete set is present.
9. Keep Preserve missing LODs from template enabled unless you intentionally replace every level.
10. Keep Use settings stored by T3 LOD Importer enabled for a direct import/export round trip.
11. The normal direct-import defaults convert Blender Z-up back to T3 Y-up and flip UV V back according to stored settings. Undo X mirror is normally disabled for direct LOD imports, but may be needed for geometry originating from a mirrored T3-ModTools glTF.
12. Export, package both SCA and LOD, and test every distance transition.

The original T-900 LOD ranges observed in the reference asset were approximately:

Level	Original triangle count	Distance range
LOD0	5,185	0 to 10
LOD1	3,637	10 to 20
LOD2	2,208	20 to 50
LOD3	831	50 to 1000

Use the target file's own ranges as the first template. A higher-detail replacement can need different budgets, but arbitrary distance changes can create popping or performance problems.

23. Animation Import and Export

Import

1. Extract a compatible rigged glTF model and the target ANM.
2. Import the glTF into Blender.
3. Use the model importer and its stored conventions. Validate an unchanged reference animation before attempting target-specific axis corrections.
4. Select the armature.
5. Use File > Import > T3 Animation (.anm).
6. Read the unmatched-bone report.

The importer stores the source FPS, `MaxTimeValue`, complete joint paths, and exact joint order on the generated Action.

Edit

- Keep the imported Action metadata.
- Preserve the source joint set for an overlay animation.
- Avoid adding unrelated armature bones to a partial upper-body action.
- Check root motion and rest pose before changing timing.
- Work on a duplicate Action.

Export

1. Make the edited Action active on the compatible armature.
2. Use File > Export > T3 Animation (.anm).
3. Keep Preserve imported ANM joint set enabled.
4. If the Action was not created by the importer, select a native reference ANM with the same role.
5. Place the exported ANM at the original internal path under the mod's `files/` folder.
6. Build and test transitions into and out of the animation, not only the isolated loop.

The exporter samples the Action and preserves the intended joint contract. Supported footstep markers can be exported as AnimationNotify events when enabled; arbitrary gameplay events are not inferred automatically.

24. T3 Transform Converter

`T3_Transform_Converter.py` provides pose analysis and editable TXT export and import through the T3 Transform Converter panel. It replaces the former bone-frame delta tool name. Chapter 35 explains single-frame reports, selected bones, reversed transforms and optional X mirroring.

This is useful for understanding or adjusting an animation numerically without treating every frame as unrelated. Rotation components use quaternions in WXYZ order. Because quaternions `q` and `-q` represent the same rotation, sign continuity matters when calculating deltas. The tool is designed to avoid false jumps caused only by that equivalent sign change and to tolerate near-zero floating-point noise.

Use it as an advanced diagnostic aid:

1. Duplicate the Action.
2. Export a short frame range for one or two bones.
3. Edit one clearly identified component delta.
4. Apply it to the duplicate Action.
5. Inspect the curve and viewport motion.
6. Export to ANM only after the edited Action remains stable.

Do not use a numerical delta report as a substitute for rig compatibility or a correct rest pose.

Part VI - Executable Patches and Advanced Research

25. Executable Patch Manifests

Ordinary asset replacement should always be preferred when it can solve the problem. Executable patches are for behavior or registration that data files alone cannot reach.

The canonical path is:

```
my_mod/
  patches/
    executable.json
```

A flat project-root executable.json remains a compatibility fallback. Prefer patches/executable.json in new projects.

aob_replace

```
{
  "format_version": 1,
  "target": "T3.exe",
  "patches": [
    {
      "id": "example_patch",
      "description": "Describe the behavior change",
      "type": "aob_replace",
      "section": ".text",
      "pattern": "8B 45 ?? 83 F8 02",
      "replacement": "8B 45 ?? 83 F8 03",
      "expected_matches": 1
    }
  ]
}
```

`??` in a pattern matches any byte. `??` in a replacement preserves the original byte. Pattern and replacement must have the same length.

aob_write

`aob_write` finds a unique anchor, validates destination bytes, and writes at a relative offset. It is used for controlled code or data injection away from the anchor.

```
{
  "id": "example_injection",
  "description": "Write validated bytes near a unique signature",
  "type": "aob_write",
  "section": ".text",
  "pattern": "AA BB CC DD",
  "write_offset": "0x20",
  "expected_fill": "00",
  "replacement": "11 22 33 44",
  "expected_matches": 1
}
```

Safety rules enforced by the tool

- Patterns are searched against the specified pristine source binary: T3.exe by default, or Ai.Dll when explicitly targeted.
- Version 1 requires exactly one expected match.
- Searches can be limited to `.text`, `.rdata`, `.data`, `.tls`, `.data1`, `.rsrc`, or `all`.
- `aob_write` must remain in the selected section.
- Expected bytes or fill are validated before writing.
- Conflicting writes stop the build.
- `T3.exe` is never overwritten.

An AOB patch is not automatically portable across releases, cracks, localized executables, or prior binary modifications. If a pattern does not match exactly once, stop and research that executable rather than weakening validation.

26. Known Limits and Research Frontiers

New content registration

File replacement is well understood. New character, weapon, vehicle, map, particle bank, or menu slots can require configuration and code paths not yet generalized by the tool.

Sound concurrency

Long automatic-weapon tails can overlap. A general rule for limiting every weapon to one active voice per actor has not been established. Design short repeated layers and test concurrency.

Terminator vision gamma

The original manual assigns Q to binocular or night vision. T3's Terminator presentation also interacts with legacy gamma behavior and `SkyNet_Gamma` settings. On modern Windows, the world tint can fail while HUD overlays remain visible. First try the game's Video options, toggle and reapply the Terminator screen option, use exclusive fullscreen, and disable external HDR, Auto HDR, night-light, or color utilities for the test. A robust modern shader replacement remains separate research.

Model export coverage

The general SCA exporter does not automatically generate map binary trees, `scene.det`, `scene.dyn`, collision targets, advanced shader parameters, lightmaps, or gameplay registration. LOD export is a separate template-based workflow.

Animation coverage

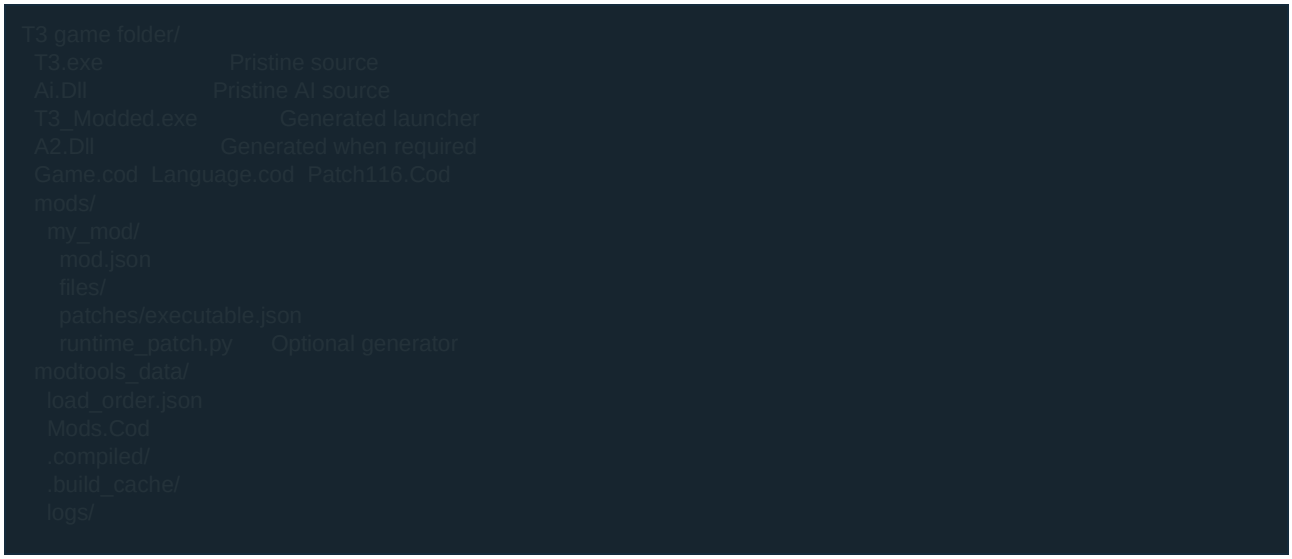
Individual animations can still require target-specific axis, root-motion, or rest-pose correction. Events tied to gameplay are not generated simply by exporting pose frames.

Renderer and shader modernization

Replacing textures and legacy material values is practical today. A generalized modern renderer, physically based material conversion, or universal post-processing pipeline is not part of the verified workflow.

Part VII - Reference

27. Folder and Path Reference



Common internal roots:

Root	Typical content
`CA/`	Character models, textures, animation libraries, and animations.
`weapons/`	Weapon option data and models.
`vehicles/`	Vehicle data and models.
`Sound11/`, `Sound22/`, `Sound44/`	Parallel sound definitions and WAV assets.
`Musics/`	OGG music cues.
`Particles/`	Particle behavior definitions.
`Textures/Particles/`	Particle textures.
`MenuDatas/3DModells/`	Menu model SCA files and their textures.
`scene/`	Map and scene data.

28. Pre-Build Checklist

- The game is closed.
- No archive tool is holding `Mods.Cod` open.
- The selected game folder directly contains the pristine `T3.exe`.
- The selected mods folder is the intended `<game>/mods/` folder.
- Every project has a valid `mod.json`.
- Every replacement is under `files/` at the exact internal path.
- Editable sources and generated native outputs are clearly separated, and all source dependencies are declared.
- Enabled projects and order are correct.
- Experimental projects are disabled unless intentionally tested.
- Executable-patch dependencies and conflicts are understood.
- The last known good project version is backed up.

29. Post-Build Checklist

- The build reports success.
- `modtools_data/Mods.Cod exists and matches the successful build report; its timestamp may remain unchanged after a valid no-op build.`
- `T3_Modded.exe and any required A2.Dll match the same successful build.`
- `modtools_data/.compiled/build_manifest.json` lists the expected enabled projects.
- The manifest lists the expected winning files.
- `modtools_data/.compiled/loader_manifest.json` lists the expected executable patches.
- The original `T3.exe` remains unchanged.
- The game is launched through `T3_Modded.exe`.
- The test covers the exact asset, several distances or contexts, and a vanilla comparison.

30. Failure-to-Cause Map

Symptom	Most likely checks
Mod is completely absent	Wrong executable, stale build, wrong mods folder, invalid project structure.
One asset is absent	Wrong internal path, higher-priority mod conflict, converted file instead of native file.
Near model works, far model is original	Missing or mismatched `model.lod`.
Texture is upside down	UV V flipped at the wrong conversion stage.
Model is mirrored	X mirror reversed twice or not reversed at export.
Faces disappear	Winding changed by a negative determinant, singular transforms, or bad normals.
Character deforms during animation	Bone names, hierarchy, rest pose, or weights do not match.
New impact DAT is ignored	The executable does not register or route to the new bank.
Build reports access denied	Game, archive viewer, editor, or security tool holds an output file.
AOB pattern fails	Different executable build, non-pristine source, or non-unique pattern.
Music never plays	Wrong cue path or the tested game state does not select that cue.
Weapon audio becomes muddy	Repeated sample tail overlap, excessive volume, or too many simultaneous emitters.

31. A Repeatable Modding Method

The most reliable T3 modding loop is simple:

1. Observe one specific behavior.
2. Extract the smallest relevant asset set.
3. Trace the exact internal path and references.
4. Back up the original and create a separate mod project.
5. Change one variable.
6. Build from the current load order.
7. Launch `T3_Modded.exe`.
8. Test in a repeatable scenario.
9. Record the result.
10. Keep the successful version before expanding scope.

This method is slower for one edit and much faster for a complete mod. It separates facts from guesses, turns failures into useful evidence, and makes discoveries reusable by the next person.

Part VIII Extended workshops for ModTools 0 6 12

This part develops complete workflows for new content rather than documenting a particular released mod. Example names such as EXAMPLE_RIFLE and ExampleUnit are placeholders, not registered game identifiers. Replace them consistently. Keep a working replacement project before attempting a new registration. T3-ModTools 0.6.12 packages assets and validates patches; it does not provide a universal Add Weapon or Add Unit wizard.

32 Adding a new weapon

Plan the identity and its dependencies

Choose a native donor weapon with the same broad behavior and a compatible animation family. Extract its effective files, checking the official patch and language overlays. Inventory the model SCA and LOD, textures and material references, weapon1.option and any weapon2.option, animation-library references, sound definitions, muzzle and projectile definitions, impact routing and menu model. Do not assume every weapon uses every file or the same number of locators.

Write a small identity sheet before editing: internal name EXAMPLE_RIFLE; asset root weapons/EXAMPLE_RIFLE/; display label; owner classes; primary and secondary behavior; animation family; existing or new sound and particle routing. Keep the display label separate from runtime names. A folder name, a weapon subtype, a sound-bank index and a menu index are different identifiers; changing one does not allocate the others.

Step 1 Prove the asset replacement

Create a project through Create mod. First replace the donor at its existing paths and confirm that the unmodified copied files work. Then edit its appearance and export a native model. For a rigged weapon, preserve the donor bone hierarchy, bind relationship and required locator IDs. Keep the editable .blend and unpacked textures inside the project, preferably in a separate source/ directory. Export native files into files/. A .blend is not loaded directly by the game.

Check the generated SCA and every LOD level before moving to registration. Confirm the texture library, submesh materials, vertex weights, bounds and muzzle position and direction. Test the menu model separately using its dedicated exporter. A correct view in Blender does not prove that the first-person view, third-person view and menu use the same axes or materials.

Step 2 Make a separate native asset family

Copy the validated donor assets to the new internal root and update references that should belong to the new weapon. Preserve intentionally shared animations or sounds rather than renaming everything blindly. Begin with donor gameplay values. Change damage, timing, ammunition and projectile behavior only after the new identity can be selected and fired.

Inspect MenuDatas/Weapons.dat and the relevant class or cast definition. Add the new entry in the same native grammar as a neighboring compatible entry and update counts where the format declares them. Add a loadout reference only to an intended owner. These data edits expose an entry but do not prove that the executable can construct or precache the new name.

Step 3 Register the runtime identity

This is advanced work. Trace how the pristine executable resolves the donor name, creates the weapon, precaches its assets, selects animations and indexes HUD or equipment tables. Record which paths are data-driven and which use fixed names, ranges or capacities. Implement a narrow name-specific adapter only where required. Preserve all original names and fallback behavior.

Never assume that appending a new row grows a fixed-size array. Check table capacity, loop bounds, allocation sizes and cleanup for every added slot. Reusing an existing compatible behavior can reduce the work, but it must not redirect unrelated weapons. New subtype numbers also require auditing particle, sound and AI consumers. There is no universally safe unused ID supplied by this handbook.

Represent validated edits in patches/executable.json, targeting the correct pristine binary. Require a unique anchor and expected destination bytes. Verify code storage, register preservation, calling convention and continuation before enabling a patch. Zero-filled bytes alone do not establish that an address is unused. Do not copy addresses or machine-code blobs from another executable build.

Step 4 Include AI and both fire modes

An entry working for the player may still be unknown to AI weapon lookup, inventory selection or aiming. Trace Ai.Dll independently when necessary; package its patches with target Ai.Dll so ModTools creates A2.Dll. Do not patch the original DLL in place. For secondary fire, verify separate option data, input handling, sound events, projectile selection and muzzle ownership. A second locator by itself does not guarantee that the engine uses it.

Acceptance exercise

Build with the smallest required dependency set. Verify selection, spawn inventory, equip and unequip, both firing modes, reload, empty ammunition, resupply, death and respawn. Repeat with an AI owner and several simultaneous shooters. Inspect FPS, TPS, menu and distant LOD views. Disable the new project and rebuild: the donor must still work and no menu entry should point to an unavailable weapon. Keep hashes and a written result for each case.

33 Adding a new playable or AI unit

Distinguish appearance from a new class

A skin replaces assets for an existing identity. A new unit needs its own identity and may also need a selectable class, AI classification and spawn eligibility. Decide which of those you actually want. Adding only a new folder or menu label does not create a new actor type. Use a compatible donor from the same faction and locomotion family to keep the first prototype small.

Step 1 Inventory a complete donor

Follow the donor cast definition under CA/casts/ into its FPS and TPS resources, model SCA and LOD, animation libraries, referenced ANMs, textures, materials and supporting character files. Include collision or ragdoll resources and soldier.human when the donor references them. Preserve native mesh and instance paths and complete bone paths. First-person and third-person rigs are separate contracts; do not transfer an animation solely because its filename looks suitable.

Step 2 Build the data identity

Create a distinct native definition, such as CA/casts/ExampleUnit.dat, and a matching resource tree using the spelling expected by its references. Copy the donor definition and change one family of references at a time. Preserve unknown fields. Check declared counts after adding or removing weapon entries. Keep the first loadout composed of already working weapons; this separates unit-registration failures from weapon-registration failures.

Use a written reference map: class or cast name, resource roots, animation libraries, allowed equipment, menu display resources, AI category and spawn rules. Search for the donor name in the extracted text files to find additional references. Do not globally replace every donor occurrence: shared animation and sound families may intentionally retain their original identity.

Step 3 Register creation and precaching

Trace the original class selection, actor construction and precache paths. A new cast may require more than one registration path, including resource initialization used before spawning. Match complete names rather than short prefixes so that the new class does not accidentally intercept an existing one. When extending tables, review every size-dependent allocation and traversal, not just the visible menu limit.

Keep player selection and bot selection separate in the investigation. Determine where Ai.Dll maps the class to movement, aiming, weapon use and other behavior. Add explicit compatibility handling only where necessary and preserve the original fallback. Patch manifests must target the binary that actually owns the behavior. ModTools builds replacement binaries from pristine sources; it does not infer these registrations from the cast file.

Step 4 Configure where the unit appears

Distinguish permission to appear from probability of appearing. A class may be restricted by faction, map, era, game mode or spawn list. An entry that can be selected manually may still be excluded from a bot population. Conversely, allowing a class does not assign it a desired population share. Trace the existing selection mechanism and test it before changing distribution.

For a population test, count enough fresh spawns to compare the new class with the original classes. Record map, mode, faction, enabled projects and the selection rule. A short sequence without the new unit is not evidence of a broken probability rule. Verify disabled or unavailable equipment cannot leave an invalid loadout or a dangling menu entry.

Step 5 Validate the whole actor

Use the native donor hierarchy as the initial baseline. Test idle, walk, run, turn, crouch if supported, aim, both fire modes, hit reactions, death and respawn. Check collisions, foot placement and ragdoll behavior separately from visible deformation. Keep SCA and LOD compatible even if a particular gameplay test appears to use only one.

Test AI following, acquiring and losing targets, firing while stationary and moving, and reacting to damage. Repeat the same scenario in the unmodified game before attributing a behavior to your new unit. Separate a locomotion issue from aim composition or weapon ownership by changing one test condition at a time. Release only after both player and AI paths have explicit results, or clearly label the unit as supported for only one of them.

34 Designing weapon audio and audiovisual effects

Build an event map before changing sounds

Find the weapon.dat definitions in Sound11, Sound22 and Sound44. List every relevant FIRE event and alternate event name, its WAV reference, volume, distance parameters and looping behavior. Verify which definition wins in the current overlay. Preserve unrelated reload, handling and mechanical events. Copy only files you intend to replace and retain the native bank structure.

Prepare a short mono PCM firing sample compatible with the donor. Use the original format as the first baseline and leave headroom. Confirm one shot before changing attenuation, equalization or engine routing. For automatic fire, compare the sample duration with the interval between shots; long repeated tails can accumulate across several actors even when each sample sounds good alone.

Create near and distant layers

Make a distant variation with less close mechanical detail and reduced high-frequency energy. Match timing so the near and far layers describe the same shot. Keep their gain relationship controlled; full-amplitude correlated layers can become louder when blended. Audition distance transitions in game rather than judging the WAVs separately.

The presence of two WAV files does not make the engine blend them. Determine whether the native definition provides the required events and attenuation or whether a validated runtime adapter is needed. If implementing an adapter, document its distance curve, per-actor state and channel lifecycle. Preserve spatial positioning and the behavior of weapons outside the intended scope.

Generate derived audio into staging, validate the samples, then publish the complete output set. Track source definitions and input audio as dependencies. Do not repeatedly use previously equalized output as the next equalization input. Exercise every supported sound bank and several simultaneous emitters. Test stopping, death, reload and weapon switching for voices that persist after their owner should stop.

Keep muzzle projectile and impact separate

Start with Particles/MuzzleFire.dat, Particles/FiredBullet.dat and the appropriate impact definitions. Follow their texture references into Textures/Particles/. Change one stage at a time and record the runtime weapon or subtype that selects it. Changing a visual texture does not automatically change dynamic light color, projectile damage or sound.

To add an independent impact bank, trace initialization, selection by weapon and surface, update, draw and release. Preserve the surface variants expected by the engine; the investigated impact system uses twelve. An added DAT is only asset data until something loads and selects it. Keep existing banks working for unrelated weapons and test primary and secondary fire independently.

Listening and visual test

Use repeatable near, middle and far positions. Fire single shots and sustained bursts indoors and outdoors, then repeat with several player and AI emitters. Compare loudness, clipping, transition continuity and overlap. Check muzzle position, projectile appearance and impact on different surfaces. Validate the disabled configuration too: original weapons and their sounds must remain functional.

35 Choosing and using the Blender plugins

Install the files from T3-ModTools/Blender Plugins as a matching set for this tool release. Use Blender's Install from Disk action, enable the required add-ons, and avoid simultaneously installing renamed duplicates of the same module. Blender and ModTools have separate version numbers; do not infer Blender compatibility from the ModTools version alone. Perform an unchanged round trip on a copied asset before a production edit.

T3 Rigged glTF Importer

T3_Blender_GLTF_Importer.py imports the converted rigged glTF or GLB and restores canonical T3 mesh and instance identities from metadata. Use File > Import > T3 Rigged glTF rather than relying only on a generic glTF import when those identities matter. For older conversions without metadata, provide the original SCA or LOD through the legacy template option. An inferred fallback is not proof of the original identifier. Bone Shape Scale changes the bone display, not the asset scale.

T3 SCA Exporter

T3_Blender_SCA_Exporter.py exports selected static or rigged meshes into native SCA text. Use it for supported replacement models, with the correct texture library and compatible armature. Preserve the imported t3_* identity properties. Review the emitted shader and texture fields; a plausible Blender preview is not a universal conversion of Blender materials. This exporter does not register new content or rebuild map binary trees.

T3 LOD Importer

T3_Blender_LOD_Importer.py imports native LOD levels, their ranges, materials, UVs and a preview skeleton. It creates a vertex group for every imported skeleton bone, including bones without a Weightmap. Those groups remain empty until deliberately weighted. Do not remove an empty group merely because no current vertices use it. Preserve the import settings and level identity properties for export.

T3 LOD Exporter

T3_Blender_LOD_Exporter.py writes edited levels into an original LOD template. Select the matching native source template and use the stored importer settings. Preserve missing levels when only part of the set is present in Blender. Verify all distance ranges and keep unedited skeleton, instance and collision data intact. A near-view edit alone is not a complete distance-tested replacement.

T3 Menu Model SCA Exporter

T3_Blender_Menu_Model_SCA_Exporter.py exports MenuDatas/3DModells assets using menu-specific rules, including the menu texture library and BASE shader workflow. Use this instead of applying world-model export settings to a posed menu mesh. Verify the exported menu asset in the selection screen; a successful in-game world-model test does not cover it.

T3 ANM Importer

T3_Blender_ANM_Importer.py applies an ANM to a compatible selected armature and stores source timing and joint information on the Action. Read unmatched-bone warnings. Keep complete hierarchy paths and distinguish partial upper-body animations from full-body animations. Import an unchanged reference first and verify its pose and timing before modifying it.

T3 ANM Exporter

T3_Blender_ANM_Exporter.py exports the active Action back to ANM. Preserve the imported joint set unless intentionally extending it. Use Add Joint to animation for selected Pose Mode bones when a new joint really belongs in that Action. The AnimationNotify panel can assign and remove left and right footstep markers and export them when enabled. It does not infer arbitrary combat or gameplay events from geometry. Test timing, transitions, missing joints and footsteps in game.

T3 Transform Converter

T3_Transform_Converter.py is the current name of the pose TXT tool. Select an armature and choose the active bone, selected bones or all bones, then set the frame range and channels. Analyze Transforms creates the report; Export Transforms as TXT saves it; Import / Apply Transform TXT applies it to a compatible target. Use a duplicate Action first. Reports use quaternion WXYZ components and can preserve quaternion sign continuity.

A single-frame range such as 0 to 0 is meaningful: it stores an absolute pose even though it has no frame-to-frame interval. Import applies that sole value. For multi-frame reports, Use TXT Start Value controls whether to start from the report or the target pose. Mirror X on Export changes exported location X and quaternion Y and Z signs without changing the scene; it does not rename left and right bones or perform general retargeting. Match the report's coordinate convention before importing. Reversed-transform commands are separate from mirroring.

T3CleverTools

T3CleverTools.py provides material self-illumination and transparency controls that the compatible export path can translate to T3 data. The ZIP is an alternative package of this tool, not a second feature set to install alongside the same module. Transparency Amount runs from opaque at 0 percent to fully transparent at 100 percent. Self-illumination changes material appearance; it does not create a dynamic light. Check the exporter and generated native fields as well as the preview, and test depth ordering and bright or dark scenes in the game.

36 Building reliable generators and dependencies

A runtime_patch.py belongs at the project root. ModTools runs enabled generators with --write using its Python interpreter, without a command shell, and imposes a 60-second limit. A generator must produce a valid executable manifest; an asset generator should declare generates_assets in mod.json so rebuilding the executable also keeps assets synchronized. Never run an untrusted generator merely because its project is called a mod.

Structure the generator as collect inputs, generate into a temporary staging directory, validate, then publish. Validate all model references, textures, manifest structure and output counts before replacing project files. The shared publish_generated_assets helper can publish files and metadata with rollback and ownership tracking, preserving unrelated authored files. Keep the .blend, source textures and configuration distinct from generated files so a regeneration cannot destroy the only editable source.

For an ordinary saved-Blender workflow, keep unpacked texture references inside the mod, or pack them in the .blend. Saving a texture without resaving the .blend must still trigger a rebuild. Declare external code and tool dependencies explicitly. A source file's existence is not proof that its current contents were included in the last successful output.

Opt into runtime_cache only for deterministic generators with complete dependencies. The entire project is hashed. dependencies adds file or directory inputs; enabled_mods tracks enabled project content and order for overlay readers; config_paths tracks paths stored in JSON configuration keys, such as a Blender executable. Exact enabled generator-folder dependencies also determine execution order. A cycle is an error, not a reason to run in an arbitrary order. This execution order does not change top-first asset priority.

Removing runtime_cache restores unconditional generator execution when dependencies cannot be represented safely. Do not use cache declarations for scripts depending on hidden network, clock or environment state. Test a warm run, a source edit, a texture edit, a dependency edit, a missing generated output and a failed generator before distributing the project.

37 Incremental builds packs and diagnostics

In 0.6.12, project sources live under mods/ while generated state lives under modtools_data/. Mods.Cod is the archive; .compiled contains build and loader reports; .build_cache contains disposable cached results; logs contains technical diagnostics. A2.Dll is generated only when the build requires AI patches. The original T3.exe and Ai.Dll remain the read-only sources.

Unchanged asset bytes can reuse validated compressed entries. A fully unchanged successful build can leave both the archive and binaries untouched. Therefore an old timestamp alone does not prove the build was skipped incorrectly. Check the console, enabled selection, winning files and relevant hashes. Content hashing catches changes that preserve a file's modification time. Repeated read-only scans share work within a phase, but final input checks are refreshed.

An input edit, load-order change, missing output or damaged cache may require work again. Updating ModTools itself can invalidate generator results. The first build after such an update can be slower. Close the game and archive viewers before rebuilding; a lock is not repaired by disabling validation. If the cache is suspected, close ModTools and remove only modtools_data/.build_cache, then build again. Do not remove mods/ or the pristine archives.

Generate mod pack combines currently enabled projects into a standalone snapshot using top-first winners and executable conflict checks. Preserve source projects separately. A generated pack is not a live link to future source changes. Review its metadata and dependencies, compile it, and test it with its source projects disabled to avoid duplicate effects. Keep any dependency that the resulting project still explicitly requires.

When a build fails, save the console message and diagnostic log. Record the selected game directory, enabled order, last edited file and original binary hashes. Use .compiled/build_manifest.json to resolve asset winners and loader_manifest.json to inspect binary patches. Do not weaken a unique-match check or expected-byte guard just to make an error disappear.

38 Testing and distributing new content

Use three milestones. First, validate structure: JSON, internal paths, native references, bone identities, LOD levels and declared dependencies. Second, validate the build: archive entries, winning providers, unchanged pristine inputs and a complete generated binary set. Third, validate behavior in game. None of these milestones replaces the others.

Start with a vanilla reproduction, then enable only the candidate and its required dependencies. Add other projects one at a time. For a weapon test player and AI use, both fire modes, equip, reload, death, respawn, sounds and all visible model contexts. For a unit include selection, permitted maps, bot population, movement, damage reactions, ragdoll and loadout availability. Rebuild and restart between configurations.

Add automated regressions for repeatable failure cases. Run the repository tests from T3-ModTools/AI Framework/Automated Tests using a suitable Python 3 interpreter. Blender-specific tests require Blender Python; an ordinary Python skip is not a Blender pass. Keep tests outside game asset directories. Check that deliberately invalid patch overlaps fail and that generator or publication failures preserve the previous complete playable output set.

For distribution include mod.json, native files, required patch manifests and any intentionally supported generator sources. Document T3-ModTools 0.6.12, dependencies, priority requirements, supported game binary fingerprints and known limitations. Include a short install, update and uninstall procedure. Do not distribute base game archives or a prepatched executable as the normal installation method. Recipients should build against their own supported pristine installation.

Keep an authoring archive with source assets and permissions, a tested release archive and a test record. A useful record lists game build, map, class, weapon, action, expected result and observed result. Mark untested AI, multiplayer or map combinations explicitly rather than implying support from a single successful local match. Retain the previous known-good release until the new one has passed the same checks.

Source Notes

This edition targets T3-ModTools 0.6.12. Technical references include the distributed README, EXECUTABLE_PATCHES specification, t3_modding.py, t3_build_cache.py, t3_generated_assets.py, the Blender plugin sources and their readmes, and regression tests in AI Framework/Automated Tests. Native paths and registration requirements should be verified against the supported pristine game installation. Example identifiers in the workshops are illustrative.

The original manual reflects the game's release era. T3-ModTools requirements and modern troubleshooting are separate from the original game's historical system requirements.

T3 Modding Guide | T3-ModTools 0.6.12 | Updated 3 November 2026